

CAPSTONE SOFTWARE DESIGN USING SOLID AND UML

PART A – APPROVED CAPSTONE TITLE

PAW-ER: An IoT-Based Pet Wellness and Safety Monitoring System with Automatic Feeding, Water Monitoring, and Mobile Application

PAW-ER is designed to help pet owners manage feeding and water supply while allowing remote pet monitoring through connected devices and a mobile application.

PART B – CLASSES, ATTRIBUTES, AND METHODS

The following are the five major classes of the PAW-ER system.

Class	Attributes	Methods
PetOwner	- ownerId : int - name : string	+ ViewPetStatus() : void + ManageFeedingSchedule() : void
FeedingService	- feedingTime : DateTime - foodAmount : double	+ DispenseFood() : void + SetFeedingSchedule() : void
WaterMonitoring	- waterLevel : double - minimumLevel : double	+ CheckWaterLevel() : void + SendLowWaterAlert() : void
CameraMonitoring	- cameraId : int - rotationAngle : int	+ RotateCamera() : void + CaptureImage() : void
NotificationService	- notificationId : int - message : string	+ SendNotification() : void + SetNotificationType() : void

PART C – UML CLASS DIAGRAM

Simple UML representation of the main PAW-ER classes and their relationships.

PetOwner
- ownerId : int - name : string
+ ViewPetStatus() : void + ManageFeedingSchedule() : void

NotificationService
- notificationId : int - message : string
+ SendNotification() : void + SetNotificationType() : void

FeedingService
- feedingTime : DateTime - foodAmount : double
+ DispenseFood() : void + SetFeedingSchedule() : void

WaterMonitoring
- waterLevel : double - minimumLevel : double
+ CheckWaterLevel() : void + SendLowWaterAlert() : void

CameraMonitoring
- cameraId : int - rotationAngle : int
+ RotateCamera() : void + CaptureImage() : void

Relationships: PetOwner is associated with the main PAW-ER services. FeedingService handles feeding, WaterMonitoring checks the water level, CameraMonitoring handles the camera, and NotificationService sends alerts.

PART D – APPLICATION OF SOLID PRINCIPLES

1. Single Responsibility Principle (SRP)

Each class has one main job. FeedingService handles feeding, WaterMonitoring handles the water level, CameraMonitoring handles the camera, and NotificationService handles notifications.

2. Open/Closed Principle (OCP)

New functions can be added without changing the main responsibility of the existing classes. For example, another notification type can be added without changing the feeding or water classes.

3. Liskov Substitution Principle (LSP)

PAW-ER monitoring components can follow a common monitoring structure. A specific monitoring component should still work correctly when the system expects a monitoring component.

4. Interface Segregation Principle (ISP)

The system can use separate interfaces for feeding, water monitoring, camera monitoring, and notifications. This prevents a class from being required to use functions that it does not need.

5. Dependency Inversion Principle (DIP)

The main services can depend on interfaces instead of directly depending on one specific device. This makes it easier to replace or add components later.

PART E – SHORT DESIGN EXPLANATION

The UML diagram shows how the main parts of PAW-ER are organized. The PetOwner interacts with the system while the other classes handle feeding, water monitoring, camera monitoring, and notifications. Separating these functions makes the system easier to understand and maintain.

PAW-ER follows the SOLID principles by giving each class a clear responsibility and allowing the system to be extended without changing unrelated parts. The design can also help when testing or improving the system in the future.

CHECKLIST

Requirement	Done
Part A – Approved Capstone Title	✓
Part B – 5 Major Classes	✓
At least 2 attributes per class	✓
At least 1 method per class	✓
Correct visibility symbols	✓
Part C – UML Class Diagram	✓
UML relationships	✓
Part D – At least 3 SOLID principles	✓
2–3 sentence explanations	✓
Part E – Short Design Explanation	✓